
Poisson Image Editing: Sparse Implementation, Applications, and Failure Analysis

Rixi Xu

Shanghai Jiao Tong University
sunny_xrxx@sjtu.edu.cn

Abstract

This report presents a rigorous replication of the Poisson Image Editing framework, implementing toy reconstruction, seamless cloning, and mixed-gradient blending. Utilizing an optimized coordinate mapping and sparse matrix, our implementation blends images within seconds. Experiments show that while Poisson editing successfully blends foreground textures and enables background penetration, it remains vulnerable to large luminance disparities and loose mask boundaries. Our codes are available at https://github.com/sunnyxrxx/AI2619_DSPDIP.

1 Paper Summary

The primary goal of local image editing is to take a selected object from a source image and insert it seamlessly into a target image. Ideally, the resulting edit should look completely natural, as if the imported object was originally photographed in the new environment. Directly copy-pasting pixels creates visible seams because of the inevitable differences in lighting and color between the two images. Classic workarounds like feathering often introduce unwanted blur or color distortion, without solving the underlying illumination mismatch. To achieve truly seamless transitions, Pérez et al. [2003] introduced a groundbreaking interpolation machinery called **Poisson Editing**.



Figure 1: Illustration of the local image editing task.

Perceptual Intuitions The motivation behind this framework is deeply rooted in how humans perceive images. Psychophysical studies [Land and McCann, 1971] show that the human visual system is highly sensitive to sharp spatial variations, like the second-order variations extracted by the Laplacian operator, whereas smooth variations are barely noticeable. Consequently, to preserve the core visual details and textures of the source object, we can enforce the second-order Laplacian of the reconstructed image f to equal that of the source image g , i.e., $\Delta f = \Delta g$ inside the edited domain Ω . Furthermore, such a Laplacian differential equation is mathematically sound once the boundary conditions are specified.

$$\Delta f = \Delta g \quad \text{over } \Omega, \quad \text{with} \quad f|_{\partial\Omega} = f^*|_{\partial\Omega}.$$

Therefore, by constraining the boundary values of f to match the target background f^* on the boundary, we get a unique and seamless transition from the target background into the edited domain.

Equivalence with Guided Interpolation On the other hand, this problem can also be interpreted through the lens of variational optimization. We can formulate our goal as aligning the first-order gradient field of f to match the gradient of the source image g as closely as possible. This can be expressed as minimizing the L_2 -norm of their gradient discrepancy

$$\min_f \iint_{\Omega} |\nabla f - \nabla g|^2 dA \quad \text{with} \quad f|_{\partial\Omega} = f^*|_{\partial\Omega}.$$

Dirichlet’s principle proves that this global minimization problem is mathematically equivalent to solving the local boundary-value differential equation defined above. Thus, these two distinct paths converge harmoniously to the same mathematical destination.

Generalization of Velocity Field While we previously treated the image gradient as a conservative velocity field, where $\text{div } \mathbf{v} = \nabla \cdot (\nabla g) = \Delta g$, this variational framework can be generalized to non-conservative velocity fields that do not necessarily correspond to the gradient of any scalar image, as formulated in Equation 1.

$$\min_f \iint_{\Omega} |\nabla f - \mathbf{v}|^2 dA \quad \text{with} \quad f|_{\partial\Omega} = f^*|_{\partial\Omega}, \quad (1)$$

whose solution is the unique solution of

$$\Delta f = \text{div } \mathbf{v} \quad \text{over } \Omega, \quad \text{with} \quad f|_{\partial\Omega} = f^*|_{\partial\Omega}. \quad (2)$$

Equation 2 is the fundamental *continuous formulation* of Poisson Image Editing.

Under this unified framework, the paper derives two core practical applications:

- **Importing Gradients:** By setting the guidance field to be the exact gradient of the source image, the imported object’s absolute colors smoothly adapt to the target background’s lighting while keeping its original textures intact.
- **Mixing Gradients:** By selectively choosing the stronger gradient between the source and target images at each point, the background’s underlying textures can seamlessly pass through the imported object, creating a semi-transparent effect.

Beyond these two core cloning paradigms, the authors leverage the remarkable versatility of the guided interpolation to propose several advanced selection editing tools.

- **Texture Flattening:** This technique passes the target gradient field through a sparse sieve to retain only the most salient structural boundaries while nullifying small-grain gradients. Solving the resulting Poisson equation washes out fine textures while preserving the primary shapes, yielding an illustrative, flattened appearance.
- **Local Illumination Changes:** Operating in the log-gradient domain, this tool applies a non-linear scaling transformation to compress large gradients and amplify small ones. This allows the system to smoothly adjust the local dynamic range, highlight under-exposed foreground objects, or mitigate specular highlights.
- **Local Color Changes:** By scaling or modifying the gradient fields across the RGB channels independently, this utility enables the recoloring of loosely selected objects while ensuring their modified boundaries smoothly and naturally blend back into the original target image.
- **Seamless Tiling:** When the editing domain is rectangular, the solver enforces periodic Dirichlet boundary conditions derived from the boundary values of the source patch. This mathematical constraint ensures that the opposite edges of the rectangle match perfectly, rendering the texture seamlessly tileable.

2 Formulation

2.1 Discrete Formulation

The *continuous formulation* of Poisson Image Editing is expressed in Equation 2. To solve it numerically, we must discretize the continuous domain and approximate the differential operators using finite differences.

Domain Discretization Let the continuous domain S be mapped to a discrete 2D grid of pixels, where each pixel is denoted by its coordinate $p = (y, x)$. For any pixel p , we define its 4-connected neighborhood N_p as the set of its immediate top, bottom, left, and right neighbors which are in S :

$$N_p = \{(y-1, x), (y+1, x), (y, x-1), (y, x+1)\} \cap S. \quad (3)$$

Difference Approximation of the Laplacian Using Taylor series expansions, the second-order partial derivatives of f at p can be approximated as

$$\Delta f_p \approx f(y, x+1) + f(y, x-1) + f(y+1, x) + f(y-1, x) - |N_p|f(y, x), \quad (4)$$

where $|N_p|$ denotes the degree of N_p . In practice, if a neighbor coordinate lies outside the physical boundaries of the image, that out-of-bounds term is omitted from the formulation.

To ensure the diagonal elements of our coefficient matrix are positive, we multiply both sides of the equation by -1 , yielding:

$$-\Delta f_p = |N_p|f_p - \sum_{q \in N_p} f_q. \quad (5)$$

Discretization of the Divergence To maintain consistency with the left-hand side, the right-hand side of Equation 2 must also be multiplied by -1 and discretized accordingly. We define the negative discrete divergence at pixel p as:

$$-\text{div } \mathbf{v}_p = \sum_{q \in N_p} v_{pq}. \quad (6)$$

Note on the Direction of Guidance Vector. In calculus, the divergence of a vector field $\mathbf{v}$ represents the **outward flux** flowing out of a region. However, since we are using the **negative divergence**, for Equation 6 to be formulated with a positive summation sign, each guidance vector v_{pq} must be defined as the **inward flux**, the explicit formulations of which are detailed in Section 2.2.

System Assembly Equating the negative discrete Laplacian in Equation 5 and the negative discrete divergence in Equation 6, we obtain the local equation for each pixel $p \in \Omega$:

$$|N_p|f_p - \sum_{q \in N_p} f_q = \sum_{q \in N_p} v_{pq}. \quad (7)$$

Since some neighbors $q \in N_p$ may lie outside the editing domain Ω , we partition the neighborhood N_p into two disjoint sets:

1. $q \in N_p \cap \Omega$: Neighbors inside the selected mask, which represent the **unknown variables** f_q to be solved.
2. $q \in N_p \cap \partial\Omega$: Neighbors outside the mask (on the boundary), whose values are **known target background values** f_q^* .

Applying this partition to the summation on the left-hand side of Equation 7, and moving the known boundary values to the right-hand side, we obtain the *discrete formulation* for each pixel $p \in \Omega$:

$$|N_p|f_p - \sum_{q \in N_p \cap \Omega} f_q = \sum_{q \in N_p \cap \partial\Omega} f_q^* + \sum_{q \in N_p} v_{pq}. \quad (8)$$

By compiling Equation 8 for all N pixels inside the mask Ω , we assemble a global system of linear equations of the form:

$$\mathbf{Ax} = \mathbf{b}, \quad (9)$$

where

- $\mathbf{x}$ is the unknown column vector containing the sought pixel values f_p for all $p \in \Omega$.
- $\mathbf{b}$ is the right-hand side vector, where each element $b[i]$ is computed using the known target boundary values f_q^* and the discrete divergence v_{pq} .
- $\mathbf{A}$ is a sparse coefficient matrix where the diagonal elements $A[i, i]$ equal the neighborhood degree $|N_p|$, and off-diagonal elements $A[i, j]$ equal -1 if the i -th and j -th pixels are adjacent within the mask Ω , and 0 otherwise.

2.2 Guidance Field and Blending Rules

The behavior of the seamless blending is dictated by the definition of the boundary conditions and the guidance field v_{pq} along the edges.

Dirichlet Boundary Conditions To guarantee a seamless transition, we enforce strict Dirichlet boundary conditions. The pixel values on the boundary $\partial\Omega$ are constrained to match the destination background image:

$$f_p = f_p^* \quad \forall p \in \partial\Omega. \quad (10)$$

Seamless Cloning (Importing Gradients) For importing an opaque object, the guidance field $\mathbf{v}$ is set directly to the gradient of the source image g , meaning the discrete edge values are defined as:

$$v_{pq} = g_p - g_q. \quad (11)$$

Mixed Gradients To blend objects with holes or transparency onto a textured background, we selectively combine the gradients of both the source image g and the target background f^* . For each oriented edge $\langle p, q \rangle$ and for each color channel independently, we select the gradient with the larger magnitude:

$$v_{pq} = \begin{cases} f_p^* - f_q^*, & \text{if } |f_p^* - f_q^*| > |g_p - g_q|, \\ g_p - g_q, & \text{otherwise.} \end{cases} \quad (12)$$

This rule ensures that high-frequency textures of the target background are preserved and seamlessly pass through flat regions of the source object.

3 Implementation Details

3.1 Pixel Indexing and Coordinate Mapping

To maximize computational and memory efficiency, we establish a 2D-to-1D coordinate mapping system using a mapping matrix **im2var** of size $H \times W$:

$$\mathbf{im2var}[y, x] = \begin{cases} i \in [0, N-1], & \text{if } \mathbf{mask}[y, x] > 0.5, \\ -1, & \text{otherwise.} \end{cases} \quad (13)$$

where N denotes the number of active mask pixels. This mapping reduces the system scale from a naive $H \times W$ to an optimal $N \times N$.

3.2 Sparse Matrix Construction

Even though the system scale has been reduced to $N \times N$, storing a full dense matrix remains inefficient because **A** is sparse, with at most five non-zero entries per row (index itself and four neighbours). To minimize memory and computational overhead, we only record the row index, column index, and the data of each non-zero entry, which can reconstruct full matrix **A** by

$$\mathbf{A}[\mathbf{rows}[k], \mathbf{cols}[k]] = \mathbf{data}[k]. \quad (14)$$

For implementation, we maintain three arrays for this coordinate format and reconstruct the Compressed Sparse Row matrix using: `A=sp.coo_matrix((data,(rows, cols)),shape=(N, N)).tocsr()`.

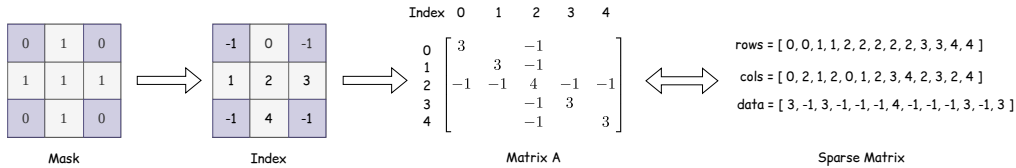


Figure 2: A demonstration of pixel indexing and sparse matrix construction.

3.3 RGB Channel Solving with Shared Matrix $\mathbf{A}$

From the above derivation, we also observe that matrix $\mathbf{A}$ only depends on the mask and is identical across all three RGB channels. Therefore, we construct the $\mathbf{A}$ exactly once. Inside the channel loop, we compute the channel-specific right-hand side vector $\mathbf{b}_{ch}$ and solve the independent systems:

$$\mathbf{A}\mathbf{x}_{ch} = \mathbf{b}_{ch}, \quad ch \in \{R, G, B\}. \quad (15)$$

This design reduces the overhead of matrix factorization and assembly by two-thirds.

3.4 Solver and Complexity

Solver We employ `scipy.sparse.linalg.spsolve` as our solver, which utilizes the highly optimized *SuperLU* direct solver¹ [Li, 2005], a sparse LU decomposition library written in C++.

Memory Complexity $O(N)$ where N is the number of mask pixels, as each row has at most 5 non-zero entries in $\mathbf{A}$. This is a dramatic reduction from $O((HW)^2)$ of dense storage without mask indexing.

Solving Complexity Sparse direct LU solvers have an average time complexity of approximately $O(N^{1.5})$ [Demmel et al., 1999], which is a massive acceleration compared to the $O(N^3)$ complexity of dense solvers.

4 Experiments

4.1 Task A: Toy Gradient Reconstruction

This experiment is a test on a single grayscale image S of size $H \times W$, whose goal is to reconstruct the image S solely from its own gradient fields G_x, G_y and its boundary pixels S^* . Under this setup, the general discrete Poisson formulation expressed in Equation 8 is specialized as follows:

- **Variables:** The role of both the source image g and the target boundary image f^* is played by the single original image S itself.
- **Domain:** The editing domain Ω is defined as the entire interior of the image grid, and the boundary $\partial\Omega$ consists of the outermost one-pixel-wide border.

Gradient Calculation We compute the horizontal and vertical forward-difference gradients of the normalized grayscale image S as:

$$G_x(y, x) = S(y, x+1) - S(y, x), \quad \forall 0 \leq y < H, 0 \leq x < W-1, \quad (16)$$

$$G_y(y, x) = S(y+1, x) - S(y, x), \quad \forall 0 \leq y < H-1, 0 \leq x < W. \quad (17)$$

These two arrays represent the absolute local changes of the image and are pre-calculated globally for reconstruction.

Linear System Construction We compile the discrete Poisson equations for all HW pixels into a global linear system $\mathbf{A}\hat{\mathbf{s}} = \mathbf{b}$ (where $\hat{\mathbf{s}}$ is the flattened vector of $\hat{S}$). For any interior pixel $p = (y, x) \in \Omega$, all its four neighbors lie inside the image grid, meaning the neighbor degree is $|N_p| = 4$ and $N_p \cap \Omega = N_p$. Under these conditions, the general discrete Poisson equation in Equation 8 naturally specializes to:

$$4\hat{S}_p - \sum_{q \in N_p} \hat{S}_q = \sum_{q \in N_p} v_{pq}. \quad (18)$$

To express the **inward gradient** v_{pq} on the right-hand side of Equation 18 using our pre-calculated forward gradients G_x and G_y , we analyze the four directed edges pointing from the center pixel

¹<https://portal.nersc.gov/project/sparse/superlu>

$p = (y, x)$ to its neighbors q :

$$v_{pq} = \begin{cases} S(y, x) - S(y, x - 1) = G_x(y, x - 1), & q = (y, x - 1), \\ S(y, x) - S(y, x + 1) = -G_x(y, x), & q = (y, x + 1), \\ S(y, x) - S(y - 1, x) = G_y(y - 1, x), & q = (y - 1, x), \\ S(y, x) - S(y + 1, x) = -G_y(y, x), & q = (y + 1, x). \end{cases}$$

Summing these four directed edge gradients yields the right-hand side of Equation 18

$$b[i] = \sum_{q \in N_p} v_{pq} = G_x(y, x - 1) - G_x(y, x) + G_y(y - 1, x) - G_y(y, x). \quad (19)$$

The matrix $\mathbf{A}$ of size $HW \times HW$ is assembled in $O(HW)$ time using coordinate triplets and solved using the SuperLU direct solver.

Boundary Conditions For pixels on the outermost border, the general Dirichlet boundary condition $f_p = f_p^*$ specializes to:

$$\hat{S}_p = S_p^*, \quad \text{for } p \in \partial\Omega. \quad (20)$$

This is implemented in our sparse system by setting the diagonal elements $A[i, i] = 1.0$, the off-diagonal elements to 0.0, and the right-hand side $b[i] = S[y, x]$. This identity equation $1 \cdot \hat{S}_p = S_p$ perfectly isolates the boundary from the interior differential equations.

4.1.1 Reconstruction Results

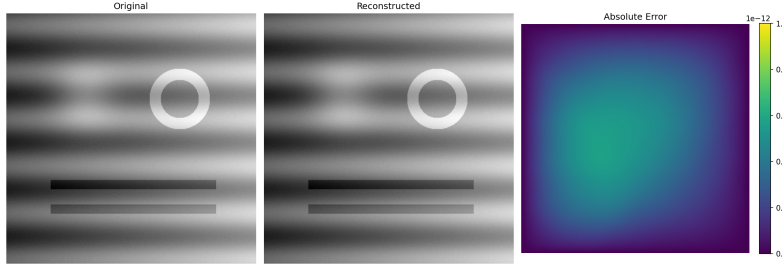


Figure 3: Reconstruction result using the source gradient and target boundary. Left: original image. Middle: reconstructed image. Right: absolute error map.

Metrics We evaluate the reconstruction accuracy using Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and Max Absolute Error, which are defined as follows:

$$\text{MSE} = \frac{1}{HW} \sum_{y=0}^{H-1} \sum_{x=0}^{W-1} (S(y, x) - v(y, x))^2, \quad \text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right), \quad (21)$$

$$\text{Max Absolute Error} = \max_{y, x} |S(y, x) - v(y, x)|, \quad (22)$$

where MAX_I denotes the maximum possible pixel value, which is set to 1 for normalized images.

The solver yields a highly precise reconstruction with:

- $\text{MSE} = 9.18 \times 10^{-26}$, $\text{PSNR} = 250.37 \text{ dB}$
- $\text{Max Absolute Error} = 5.72 \times 10^{-13}$

This tiny error is purely attributed to the double-precision floating-point round-off limit of the computer.

4.2 Task B: Seamless Cloning

Figure 4 shows the source, mask, and target image used in our experiment. All of them have the same size, and the source image is blended with the target image with no positional offset.

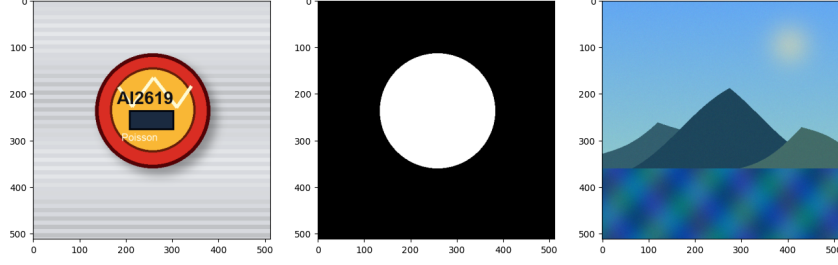


Figure 4: The source image, mask, and target image.

4.2.1 Overall Naturalness Analysis

Figure 5 presents a comparison between the naive copy-and-paste baseline and the Poisson seamless cloning implementation.

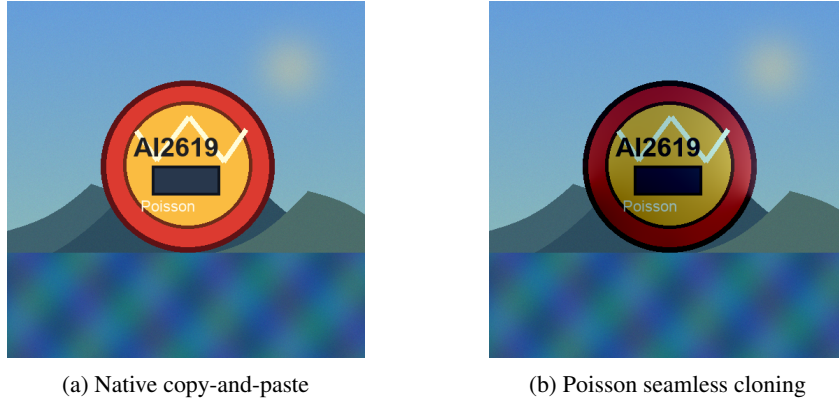


Figure 5: Comparison of local image blending results. (a) Naive copy-and-paste exhibits a prominent seam and unnatural brightness. (b) Poisson seamless cloning eliminates the seam while preserving its internal high-frequency textures.

As shown in Figure 5a, naive pasting introduces a harsh boundary seam due to the raw pixel intensity mismatch between the source’s bright background and the target’s blue sky and dark mountains. Conversely, Figure 5b demonstrates that our Poisson cloner successfully eliminates this seam. Furthermore, the colors of the cloned badge dynamically adapt to the target environment’s illumination. Specifically, the red outer ring naturally darkens at the bottom to match the shaded mountain ridge, while the yellow interior smoothly transitions to align with the warm sky illumination, all while keeping the high-frequency text "AI2619" perfectly intact and legible.

4.2.2 Boundary Analysis

Figure 6 provides a localized magnification of the boundary transition where the cloned badge overlaps with both the bright sky and the dark mountain ridge simultaneously.

In the naive copy-paste result, the imported red and dark-brown boundary rings of the badge remain statically uniform in color, failing to adapt to the dramatic transition of the background. In contrast, the Poisson cloning result exhibits an elegant, spatially varying adaptation along the circular boundary. This is because Dirichlet boundary conditions enforce pixel-value continuity at the region edge, aligning the source boundary with the target background’s change from blue sky to dark mountains.

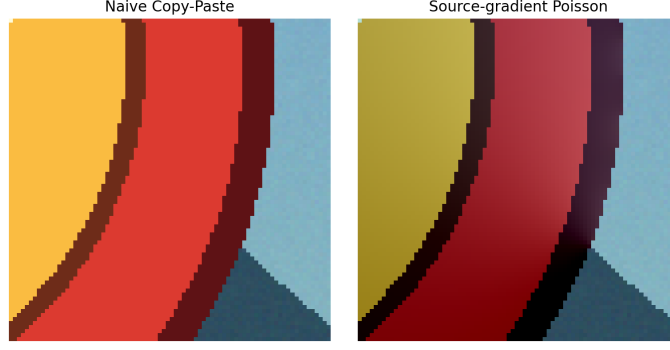


Figure 6: Comparison of the boundary transitions. Left: naive copy-paste exhibits static colors. Right: Poisson cloner dynamically adapts the boundary to match the background.

4.3 Task C: Mixed Gradients

4.3.1 Overall Comparison

We also implement the mixed-gradient blending in Equation 12 using the same example shown in Figure 4.

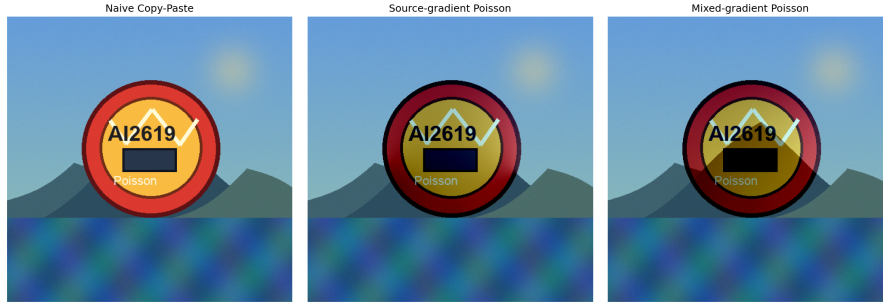


Figure 7: Comparison between copy-and-paste (left), source-gradient blending (middle) and mixed-gradient blending (right). The source-gradient approach erases the target background while the mixed-gradient approach allows background to penetrate the yellow region of the badge.

Figure 7 illustrates the qualitative difference between source-gradient Poisson blending and mixed-gradient Poisson blending.

In the source-gradient result, no trace of the target background f^* is preserved within the mask region Ω , and the background mountains are completely overwritten by the opaque flat textures of the badge.

The mixed-gradient method creates a semi-transparent overlay where the background mountain ridge **penetrates the badge**. This is because the sharp silhouette of the target's mountain ridge presents a steeper gradient than the flat region of the source badge, and the Poisson solver selects and reconstructs the background mountain texture inside the edited region.

4.3.2 Micro-level Comparison

Figure 8 presents two localized comparisons among the three paradigms.

We discover that the internal colors in the mixed-gradient result (right) are significantly **more speckled and fragmented** than the former two paradigms. Instead of presenting flat blocks of uniform red or yellow, the mixed-gradient image contain randomly distributed pixels of closely related shades. This is a direct consequence of the non-linear gradient selection rule as in Equation 12, because the mixed-gradient operator selectively imports the target background's high-frequency noise into the mathematically flat, zero-gradient regions of the source badge.

In summary, the mixed-gradient approach provides an powerful mechanism for image compositing. By allowing salient background textures to **seamlessly penetrate** flat source regions while keeping **sharp foreground contours crisp**, the solver successfully prevents the artificial sticker-like appearance of standard cloning. Furthermore, because background features like the mountain silhouette cross the selection border continuously without any interruption, the boundary naturalness is greatly enhanced, transforming a simple cut-and-paste into a visually convincing, coherent physical scene.

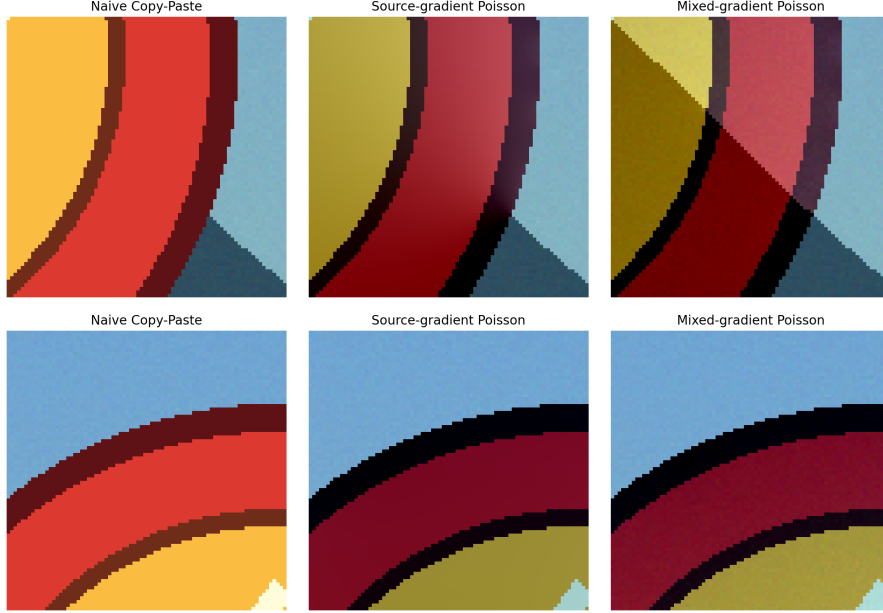


Figure 8: Comparison of the boundary transitions.

4.4 Runtime

Table 1 shows the runtime results across different experiments. The matrix scale is $N \times N$, where N represents the number of unknowns. Number of Non-Zeros (NNZ) entries only accounts for less than 0.1% for such a huge matrix.

Table 1: Quantitative performance metrics and system statistics across different experiments. All results are averaged over six independent runs on a single CPU core. The primary solver used is SuperLU via `scipy.sparse.linalg.spsolve`.

Experiment	Unknowns	Matrix Scale	NNZ	Runtime (seconds)		
				Build	Solve	Total
Toy Reconstruct	262,144	$262,144 \times 262,144$	1,304,584	0.7635	2.8262	3.5897
Seamless Cloning	48,283	$48,283 \times 48,283$	240,423	0.2325	1.4430	1.6755
Mixed Gradients	48,283	$48,283 \times 48,283$	240,423	0.2374	1.4926	1.7300

5 Failure Analysis

Although Poisson Image Editing is highly effective under normal conditions, it suffers from inherent mathematical limitations. This section analyzes two common failure modes of gradient-domain editing using the quantitative data and visualizations generated in our experiments.

5.1 Case 1: Brightness Scaling

Experimental Phenomena As shown in Figure 9, the $1.8\times$ scaled brightness image, the text "Poisson" and the wavy line boundaries become highly blurred and merge into the surrounding background, losing their original contrast and sharp edges. Additionally, the interior yellow region of the badge undergoes a severe color shift, turning into a dull yellow-green color.

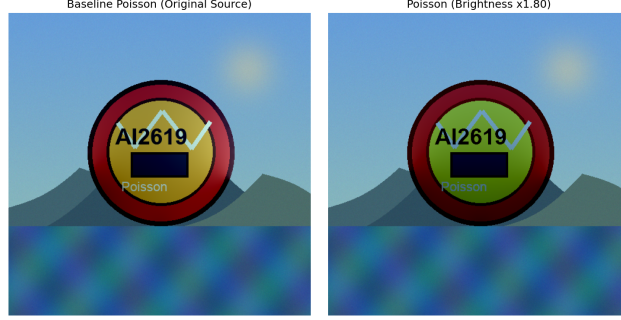


Figure 9: Comparison of seamless cloning result before and after brightness scaling. Left: original source image. Right: scaled brightness image.

Reason Analysis Many works have discussed the color bleeding problem from the perspective of gradient guidance and boundary conditions [Afifi and Hussain, 2015, Qin et al., 2008]. The degradation stems from **asymmetric gradient attenuation** across different channels and regions due to numerical saturation clamping. In both the baseline and scaled experiments, the Dirichlet boundary conditions of the system remain **identical**, meaning the numerical starting point of the reconstruction is consistent and depends solely on the target image. Theoretically, scaling the source image by $1.8\times$ should amplify the gradients by the same factor. However, because of the strict $[0.0, 1.0]$ intensity limit, high-brightness pixels saturate and **get clamped**, which actually annihilates the gradient field.

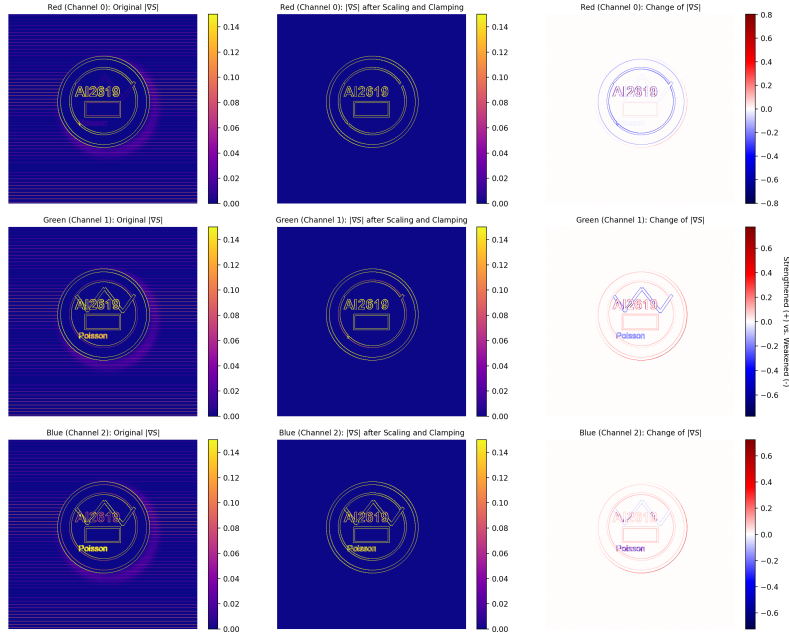


Figure 10: Gradient attenuation after brightness scaling, compute in R, G, B channel respectively. For the last column, red for increase in gradient and blue for decrease.

As shown in the first row of Figure 10, the Red channel gradients along the boundary of the yellow disk are almost entirely deep blue, indicating severe attenuation. This reduces the guidance force of

the Red channel, making the spatial pixel transitions slow. Meanwhile, the Green channel undergoes much less attenuation and receives stronger support from the green-colored mountain background. This asymmetric degradation across channels destroys the original warm yellow ratio of the source badge and shifts the yellow center to a dull yellow-green.

Furthermore, the text "Poisson" is a high-frequency detail. In the second and third rows of the change map, the text region is entirely deep blue, which indicates a massive reduction in gradients. Consequently, during the reconstruction process, the solver flattens the transition edge between the text and the yellow background, leading to a severe loss of visual contrast.

5.2 Case 2: Mask Dilation

Experimental Phenomena A prominent circular halo consisting of horizontal gray stripes appears around the badge, creating an unnatural ghosting and blurry transition zone along the boundaries of the blended region.

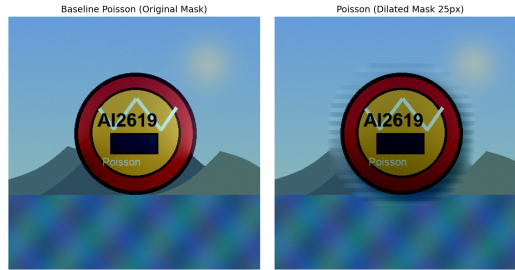


Figure 11: Comparison of seamless cloning under a dilated mask. Left: original tight mask. Right: dilated mask.

Reason Analysis Within this 25-pixel wide transition zone, the source image contains a gray-striped background. Because the Poisson equation inside the selection strictly enforces gradient matching, the solver is forced to reconstruct these horizontal gray-striped gradient features, which pollutes the gradient field of the transition region. At the outermost boundary, the solver must seamlessly transition from these reconstructed gray stripes to the target blue sky and mountains. Since the gray background of the source image and the target background are highly inconsistent in color and brightness, the solver performs a smooth interpolation to eliminate this large color difference. The spatial propagation of this high-frequency error manifests visually as a semi-transparent, gray horizontal-striped halo [Jia et al., 2006].

6 Conclusion

This paper evaluated the mathematical properties, implementation details, and limitations of Poisson Image Editing through comparative experiments.

The fundamental difference between Poisson editing and traditional naive copy-paste lies in the operating domain. Naive pasting operates in the pixel domain and creates harsh boundary seams due to numerical discontinuities. In contrast, Poisson editing operates entirely in the gradient domain. By solving the discrete Poisson equation under Dirichlet boundary conditions, it guarantees absolute boundary continuity and smoothly interpolates the target illumination into the interior. Mathematically, because the Poisson formulation is a strictly convex quadratic optimization problem, it guarantees a unique global minimum and is entirely free from local minima, ensuring high numerical robustness.

Despite its advantages, the algorithm exhibits three distinct physical limitations. First, because the formulation lacks a mechanism to preserve absolute colors, extreme boundary intensity mismatches propagate globally, causing severe color bleeding and hue shifts. Second, the algorithm is highly sensitive to mask boundaries. Dilating the mask introduces unwanted background gradients from the source image, forcing the solver to integrate irrelevant details and creating prominent halo artifacts. Finally, being a purely 2D interpolation framework, Poisson editing cannot synthesize 3D perspective, specular highlights, or directional shadows, which can still lead to physical inconsistencies under conflicting lighting conditions.

AI Usage Declaration

Generative AI models (Gemini 3.5 and GPT 5.2) were utilized during this project as auxiliary tools. Specifically, they were employed for code debugging, formulating $\text{\LaTeX}$ mathematical typesetting, and proofreading the English syntax of this report. **No automatic image-editing or neural network models were used to substitute any of the empirical results or physical experiments reported herein.**

References

- Patrick Pérez, Michel Gangnet, and Andrew Blake. Poisson image editing. *ACM Transactions on Graphics*, 22(3):313–318, 2003.
- Edwin H Land and John J McCann. Lightness and retinex theory. *Journal of the Optical society of America*, 61(1):1–11, 1971.
- Xiaoye S Li. An overview of superlu: Algorithms, implementation, and user interface. *ACM Transactions on Mathematical Software (TOMS)*, 31(3):302–325, 2005.
- James W Demmel, Stanley C Eisenstat, John R Gilbert, Xiaoye S Li, and Joseph WH Liu. A supernodal approach to sparse partial pivoting. *SIAM Journal on Matrix Analysis and Applications*, 20(3):720–755, 1999.
- Mahmoud Afifi and Khaled F Hussain. Mpb: A modified poisson blending technique. *Computational Visual Media*, 1(4):331–341, 2015.
- Chuan Qin, Shuozhong Wang, and Xinpeng Zhang. Image editing without color inconsistency using modified poisson equation. In *2008 International Conference on Intelligent Information Hiding and Multimedia Signal Processing*, pages 397–401. IEEE, 2008.
- Jiaya Jia, Jian Sun, Chi-Keung Tang, and Heung-Yeung Shum. Drag-and-drop pasting. *ACM Transactions on graphics (TOG)*, 25(3):631–637, 2006.